

THE 12-MONTH LLM ENGINEER ROADMAP

FAANG-Level Prep • From DSA in C++ to LLM Engineering

A complete, week-by-week guided plan covering DSA, OOPs, OS, DBMS, Computer Networks, System Design, ML/DL, and LLM Engineering — built for a full-time college student, with dedicated tracks for notes, typing speed, behavioral prep, and interview readiness.

12 Months | 4 Phases | 48 Weeks | 9 Projects

Table of Contents

1. How to Use This Roadmap	3
2. Daily & Weekly Time Budget	3
3. Roadmap at a Glance — 4 Phases	4
4. Phase 1 (Months 1-3): Foundations — C++, OOPs, Core DSA	4
5. Phase 2 (Months 4-6): DSA Depth + OS + DBMS + CN	5
6. Phase 3 (Months 7-9): System Design + ML/DL Foundations	6
7. Phase 4 (Months 10-12): LLM Engineering + FAANG Interviews	7
8. Project Portfolio — Detailed Specs	8
9. DSA Pattern Cheat Sheet	9
10. System Design Building-Blocks Glossary	10
11. OS / DBMS / CN Quick-Revision Cheat Sheet	11
12. Note-Making System	12
13. Typing Speed Improvement Plan	12
14. Behavioral & HR Interview Prep	13
15. Company-Wise Interview Focus	13
16. Resume, GitHub & Application Strategy	14
17. Resource Library (Subject-wise)	14
18. Common Pitfalls to Avoid	14
19. Monthly KPI Tracker	15

1. How to Use This Roadmap

This plan assumes you are attending college full-time, so it is built around **25-30 focused hours/week**, not 60. It is intentionally aggressive — reaching "FAANG-level" in 12 months while starting from DSA basics is possible but requires near-daily consistency. Treat exam weeks as buffer weeks (each month has slack built in); shift, don't skip.

- **Sequencing rule:** DSA and typing practice run every single week of all 12 months — they are never "done," only leveled up. Everything else (OOPs, OS, DBMS, CN, System Design, ML/LLM) is layered in phase by phase so you're never juggling more than 2 new subjects at once.
- **Weekly structure:** every month below is broken into 4 weeks — 3 "learning" weeks and 1 "consolidation" week (revision + project + mock test). Follow this rhythm even if you fall slightly behind — protect the consolidation week, it's what prevents topics from evaporating.
- **Projects are checkpoints, not homework:** each one forces you to apply that month's theory before moving on. Do not start the next month's new topic until the current project at least "works end-to-end," even if rough.
- **Revise on a rolling basis:** use the Sunday revision ritual (Section 12) every week, not just at month-end.
- **Track everything:** use Section 19's KPI tracker at the end of every month — 5 minutes, non-negotiable.
- **Difficulty ramps intentionally:** Phase 1 problems are Easy-heavy, Phase 2 mixes in Medium, Phase 3-4 are Medium/Hard-heavy with timed constraints to simulate real interviews.
- **If you slip a week:** don't try to "catch up" by cramming two weeks into one. Compress the least critical sub-topic (usually the deepest edge-case theory) and keep the rhythm intact.

2. Daily & Weekly Time Budget

Day	Time Slot	Focus
Mon-Fri	20-25 min (morning/commute)	Typing practice + Anki flashcard review
Mon-Fri	2-3 hrs (evening, after college)	DSA problems (1 new topic/pattern) or current subject theory, alternating days
Sat	6-7 hrs	Deep work block: new theory + hands-on project time (split 60/40)
Sun	5-6 hrs	Project work + weekly revision ritual + notes clean-up + 1 mock test/contest

Total $\approx$ 27-30 hrs/week. Sample weekday evening breakdown: 15 min warm-up (re-solve yesterday's hardest problem from memory) $\rightarrow$ 90 min new problems/theory $\rightarrow$ 30-45 min notes + Anki card creation. Adjust proportionally if college workload is heavier some weeks — protect the daily typing + DSA habit above all else since consistency compounds more than any single long session.

Sample Hour-by-Hour Weekday (reference template)

Time	Activity
7:00-7:20 AM	Typing practice (monkeytype.com) + 10 Anki cards
College hours	Attend classes — use breaks for 1 quick flashcard pass on phone
6:30-7:00 PM	Warm-up: re-derive yesterday's hardest solution from scratch
7:00-8:30 PM	New DSA problems or subject theory (OS/DBMS/CN/SD/ML per current month)
8:30-9:00 PM	Convert today's learning into notes + 3-5 new Anki cards
9:00-9:15 PM	Log progress in KPI tracker (only on consolidation-week Sundays, otherwise skip)

3. Roadmap at a Glance — 4 Phases

Phase	Months	Core Focus	Milestone Project(s)	DSA Cumulative Target
Phase 1	1-3	C++, OOPs, Core DSA (arrays $\rightarrow$ trees), SQL basics, note system + typing setup	OOP console app, SQL mini-project	~230-250
Phase 2	4-6	DSA (graphs, DP), Operating Systems, DBMS depth, Computer Networks, Python intro	CPU scheduler, concurrency simulator, socket chat app	~300-320
Phase 3	7-9	LLD + HLD System Design, ML/DL foundations, NLP & Transformers	LLD case study, HLD case studies, NLP classifier	~420-450
Phase 4	10-12	LLM Engineering (RAG, fine-tuning, agents, deployment), mock interviews, applications	RAG chatbot, fine-tuned LLM, capstone LLM app	~500+

4. Foundations — C++, OOPs, Core DSA

Month 1 — C++ Fundamentals + OOPs + DSA Basics

Week	Focus & Topics	Target
Week 1	C++ core syntax: variables, data types, operators, control flow, functions & parameter passing (value/reference/pointer), I/O streams, intro STL (<code>vector</code> , <code>pair</code> , <code>string</code>)	10 problems
Week 2	OOPs I: classes & objects, constructors (default/parameterized/copy), destructors, <code>this</code> pointer, static members, access specifiers, encapsulation. DSA: complexity analysis (Big-O/Θ/Ω), recursion (factorial, fibonacci, power, GCD, sum of digits)	15 problems
Week 3	OOPs II: inheritance (single/multiple/multilevel/hierarchical), polymorphism (overloading vs overriding, virtual functions & vtables), abstraction (pure virtual/interfaces), operator overloading, friend functions. DSA: array patterns (rotation, Kadane's max subarray, prefix sums, Dutch national flag), string basics (palindrome, anagram, reverse words)	20 problems
Week 4	Consolidation: revise all OOPs concepts via a self-quiz; solve 20 mixed easy problems; set up Obsidian/Notion vault + Anki deck; install a typing app and record baseline WPM	20 problems

Project: Console-based Library/Bank Management System in C++ — menu-driven, uses inheritance for user roles (Admin/Member), file handling for data persistence, and at least one operator overload (e.g., comparing account balances).

Also this month: Typing target 30 WPM by month-end. Create your first 20 Anki cards (OOPs definitions + complexity classes).

Month 2 — DSA: Linear Structures + Hashing + Patterns

Week	Focus & Topics	Target
Week 1	Linked Lists: singly, doubly, circular; insert/delete/reverse (iterative & recursive), cycle detection (Floyd's), merge two sorted lists, find middle node	20 problems
Week 2	Stacks & Queues: array/linked implementation, monotonic stack, queue-using-stacks, circular queue, deque. Problems: valid parentheses, next greater/smaller element, sliding window maximum (intro)	20 problems
Week 3	Hashing: hash maps/sets, collision handling (chaining/open addressing concept), frequency counting, two-sum family, subarray-sum-equals-k. OOPs add-on: Singleton & Factory design patterns (concept + C++ code)	20 problems
Week 4	Consolidation: Sorting (merge sort, quick sort, counting sort) & searching (binary search + first/last occurrence + search in rotated sorted array); two-pointer & sliding window intro; join 1 timed contest (Codeforces Div 3/4 or LeetCode Weekly)	20 problems

Typing target: 35–40 WPM. **Notes:** start an "OOPs + DSA Patterns" Anki deck — one card per pattern with trigger keywords (e.g., "sorted array + target" → binary search).

Month 3 — DSA: Trees, Heaps, Tries, Intro Graphs + DBMS Basics

Week	Focus & Topics	Target
Week 1	Binary Trees: traversals (inorder/preorder/postorder — recursive & iterative), level-order (BFS), height, diameter, balanced-tree check, path-sum problems	18 problems
Week 2	BST: insert/delete/search, validate BST, kth-smallest, LCA in BST, BST-iterator. Heaps: min/max heap, <code>priority_queue</code> , kth largest/smallest, heap sort. Tries: insert/search/prefix, word search II (intro)	20 problems
Week 3	Graphs intro: adjacency list/matrix representation, BFS, DFS, connected components, cycle detection (directed & undirected). DBMS starts: ER modeling, relational model, keys (primary/foreign/candidate/composite/super)	18 problems
Week 4	Consolidation + DBMS: normalization (1NF→BCNF with worked examples), SQL joins/subqueries/GROUP BY/HAVING, ACID properties, transactions & isolation levels. Build the DBMS project	15 problems

Project: Student/Inventory Management System — C++ or Python backend + MySQL/PostgreSQL. Features: normalized 3NF schema, CRUD via prepared statements, at least 2 multi-table JOIN reports, a stored procedure or trigger for stock/attendance updates.

Milestone check: ~230–250 cumulative DSA problems, OOPs fully internalized, first SQL project shipped. Typing target: 45 WPM.

5. DSA Depth + Operating Systems + DBMS + Computer Networks

Month 4 — Graphs Advanced + DP Intro + Operating Systems

Week	Focus & Topics	Target
Week 1	Shortest paths: Dijkstra, Bellman-Ford, Floyd-Warshall (intro). MST: Prim's & Kruskal's. OS: process vs thread, process states, PCB, context switching	15 problems
Week 2	Topological sort (Kahn's + DFS-based), cycle detection revisited, bipartite check. OS: CPU scheduling — FCFS, SJF (preemptive/non-preemptive), Round Robin, Priority scheduling, scheduling metrics (turnaround/waiting/response time)	15 problems
Week 3	DP intro: 1D DP (climbing stairs, house robber, fibonacci-memoization), understanding memoization vs tabulation. OS: multiprogramming vs multitasking, basics of memory management (contiguous allocation, fragmentation)	15 problems
Week 4	Consolidation: mixed graph + DP revision; build the CPU scheduler project; 1 timed contest	15 problems

Project: CPU Scheduling Simulator in C++ — implements FCFS/SJF/Round Robin, takes arrival & burst times as input, outputs a Gantt chart (ASCII or simple graphic) plus average waiting/turnaround time per algorithm for comparison.

Month 5 — DP Advanced + OS Depth + Python Kickoff

Week	Focus & Topics	Target
Week 1	2D DP: knapsack (0/1, unbounded), coin change, LCS, edit distance, matrix path problems. OS: deadlocks — 4 necessary conditions, prevention/avoidance, Banker's algorithm (worked example)	15 problems
Week 2	LIS & variants, DP on strings (palindromic substrings, wildcard matching). OS: process synchronization — race conditions, critical section, mutex vs semaphore, producer-consumer & readers-writers problems	15 problems
Week 3	Greedy algorithms (activity selection, job sequencing, fractional knapsack) + backtracking (N-Queens, subsets/permutations, Sudoku solver intro). OS: virtual memory, paging, segmentation, page replacement (FIFO/LRU/Optimal), thrashing	15 problems
Week 4	Consolidation + Python kickoff: Python syntax, OOP in Python, list/dict comprehensions, NumPy & Pandas basics — the language for all upcoming ML/LLM work. Build the concurrency project	15 problems

Project: Producer-Consumer simulation using threads + semaphores/mutex (C++ or Python) with a bounded buffer, demonstrating race-condition prevention with clear logging of thread states.

File systems (allocation methods, inodes, directory structures) — fold into your OS Anki deck this week alongside virtual memory.

Month 6 — DSA Revision + Bit Manipulation + Computer Networks

Week	Focus & Topics	Target
Week 1	Bit manipulation (AND/OR/XOR tricks, count set bits, power of two, single number), Union-Find (DSU with path compression + union by rank). CN starts: OSI vs TCP/IP model, encapsulation/de-encapsulation	15 problems
Week 2	Segment trees & Binary Indexed Trees (Fenwick) — range sum/min queries, point updates. CN: HTTP vs HTTPS, DNS resolution flow, TCP vs UDP, three-way handshake, congestion control (intuition)	15 problems
Week 3	Full mixed-topic revision pass across Months 1-5 (timed, interview-style); CN: routing basics (static/dynamic), NAT, subnetting intuition, socket programming API (bind/listen/accept/connect)	15 problems
Week 4	Consolidation: build the chat app project; take a full mixed-topic mock test (60-90 min, 3 problems)	15 problems

Project: Client-server chat application using TCP sockets (C++ or Python), supporting multiple concurrent clients via threads/select(), with usernames and a broadcast/whisper feature.

Milestone check: ~300-320 cumulative DSA problems, OS + DBMS + CN core theory complete, Python comfortable. Typing target: 60 WPM.

6. System Design + ML/DL Foundations

Month 7 — Low-Level Design (LLD) + Math for ML + Neural Nets Intro

Week	Focus & Topics	Target
Week 1	SOLID principles (with before/after code examples), UML basics (class diagrams, sequence diagrams). DSA: Medium-level graph/DP mixed revision	12 problems
Week 2	Design patterns I: Factory, Abstract Factory, Singleton, Builder — when to use each, C++ implementations. Math for ML: vectors, matrices, matrix multiplication, dot/cross product intuition	12 problems
Week 3	Design patterns II: Observer, Strategy, Adapter, Decorator. Math for ML: probability basics, Bayes' theorem, distributions, mean/variance/std-dev, gradient/derivative intuition	12 problems
Week 4	Consolidation: Neural network basics — perceptron, activation functions, forward pass, backpropagation derivation, gradient descent (batch/SGD/mini-batch); PyTorch tensors & autograd hands-on	10 problems

Project: LLD case study — design and fully code a Parking Lot or Elevator System applying at least 3 design patterns (e.g., Strategy for pricing, Observer for notifications, Factory for vehicle types), with class diagrams saved in your notes.

Month 8 — High-Level Design (HLD) + Deep Learning + Intro NLP

Week	Focus & Topics	Target
Week 1	HLD foundations: scalability (vertical vs horizontal), load balancing algorithms, caching strategies (write-through/write-back, cache eviction), CDN basics	10 problems
Week 2	CAP theorem, SQL vs NoSQL trade-offs, database sharding & replication, consistent hashing. DL: CNN intuition (convolution, pooling, common architectures overview)	10 problems
Week 3	Message queues (Kafka/RabbitMQ concepts), microservices vs monolith, API gateway, rate limiting algorithms (token bucket, sliding window). DL: RNNs, LSTMs, vanishing gradient problem	10 problems
Week 4	Consolidation: NLP basics — tokenization (word/subword/BPE), embeddings, Word2Vec/GloVe intuition (skip-gram vs CBOW). Write full HLD docs for 2 case studies	10 problems

Project: Full HLD design documents for 2 classic systems (e.g., URL Shortener and Instagram-style feed) — including capacity estimation, high-level architecture diagram, data model, and explicit trade-off discussion. Portfolio-ready writeups.

Month 9 — Transformers + Hugging Face + Mock System Design

Week	Focus & Topics	Target
Week 1	Attention mechanism from scratch (Q/K/V, scaled dot-product), self-attention vs cross-attention, multi-head attention	10 problems
Week 2	Full Transformer architecture (encoder-decoder), positional encoding; BERT (encoder-only, masked LM) vs GPT (decoder-only, causal LM) architectural differences	10 problems
Week 3	Hugging Face ecosystem hands-on: transformers , datasets , tokenizers libraries; load a pretrained model and run inference. 1st mock system design interview (peer or platform)	10 problems
Week 4	Consolidation: fine-tune a small pretrained model for classification; 2nd mock system design interview; shift DSA to Medium/Hard-only pattern revision	10 problems

Project: Fine-tune DistilBERT (or similar) for text classification/sentiment analysis using Hugging Face's Trainer API — your first real NLP project, with an evaluation report (accuracy/F1/confusion matrix).

Milestone check: ~420–450 cumulative DSA problems, can explain attention mechanism from scratch on a whiteboard, LLD + HLD portfolio has 3 written case studies. Typing target: 70–75 WPM.

7. LLM Engineering + FAANG Interview Prep

Month 10 — Core LLM Engineering: Fine-Tuning + RAG

Week	Focus & Topics	Target
Week 1	Prompt engineering fundamentals: zero-shot, few-shot, chain-of-thought, ReAct prompting, system vs user prompts	8 problems
Week 2	Parameter-efficient fine-tuning: LoRA (intuition + math), QLoRA (quantization + LoRA), Hugging Face PEFT library hands-on	8 problems
Week 3	RAG architecture: embeddings, chunking strategies, vector databases (FAISS, Chroma, Pinecone) — indexing & similarity search (cosine/dot-product)	8 problems
Week 4	Consolidation: LangChain/LlamaIndex framework basics — chains, retrievers, memory; assemble the RAG project; 2 timed mixed-topic DSA mock tests	10 problems

Project: RAG-based Q&A chatbot over a custom document set — chunking + embedding pipeline, vector DB retrieval, LLM API for generation, and a simple chat UI (Streamlit/Gradio).

Month 11 — Agentic Systems + MLOps + Mock Interviews Begin

Week	Focus & Topics	Target
Week 1	Agentic workflows: tool-calling agents, LangGraph/AutoGen concepts, planning & multi-step reasoning loops. Start 1 DSA mock/week	6 problems
Week 2	LLM evaluation methods (BLEU/ROUGE limitations, LLM-as-judge), hallucination mitigation, guardrails/safety filters. Start 1 system design mock/week	6 problems
Week 3	Deployment: FastAPI wrapper for model serving, Docker containerization, basic cloud deploy (AWS/GCP free tier), quantization/vLLM concepts for latency. Resume overhaul begins	6 problems
Week 4	Consolidation: fine-tune + deploy the LLM project end-to-end; GitHub & LinkedIn polish; 1 behavioral mock (STAR method)	6 problems

Project: Fine-tune an open-source LLM (e.g., Llama 3 8B or Mistral 7B) with LoRA on a custom task/dataset; wrap it in a FastAPI endpoint and containerize with Docker for reproducible deployment.

By now you should be doing 2-3 mock interviews/week: 1 DSA, 1 system design, occasional behavioral.

Month 12 — Capstone + Full Interview Loop + Applications

Week	Focus & Topics	Target
Week 1	Capstone architecture & build begins (multi-agent RAG assistant or AI coding/reasoning assistant); full interview loop practice starts (2 DSA + 1 SD + 1 ML round/week)	6 problems
Week 2	Capstone core features complete; rapid revision of OS/DBMS/CN/OOPs using cheat sheets + Anki only (no new material); behavioral question bank practice	6 problems
Week 3	Capstone deployed + documented + demo video recorded; portfolio site built linking all 9 projects; begin tailored applications	6 problems
Week 4	Consolidation: final mock-interview loop; polish resume per role; track applications; light DSA maintenance only	6 problems

Typing target: 80-90 WPM. **End-state:** ~500+ DSA problems solved, 9 deployed/documented projects, full interview-loop practice completed, applications in flight.

8. Project Portfolio — Detailed Specs

1. Library / Bank Management System Month 1

Tech: C++, file I/O. **Features:** role-based menus (Admin/Member) via inheritance, add/issue/return records, persistent storage in files, input validation, operator overloading for record comparison. **Demonstrates:** OOP fundamentals, encapsulation, file handling.

2. Student / Inventory Management System Month 3

Tech: C++/Python + MySQL/PostgreSQL. **Features:** normalized 3NF schema, CRUD via prepared statements, multi-table JOIN reports, a trigger/stored procedure for auto-updates. **Demonstrates:** DBMS design, SQL proficiency.

3. CPU Scheduling Simulator Month 4

Tech: C++. **Features:** FCFS/SJF/Round Robin implementations, Gantt chart output, average waiting/turnaround time comparison across algorithms. **Demonstrates:** OS scheduling concepts.

4. Producer-Consumer Simulation Month 5

Tech: C++/Python threads + semaphores/mutex. **Features:** bounded buffer, race-condition-free logging of thread states, configurable producer/consumer counts. **Demonstrates:** OS concurrency & synchronization.

5. Socket-Based Chat Application Month 6

Tech: C++/Python, TCP sockets. **Features:** multiple concurrent clients (threads/select()), usernames, broadcast + whisper messaging. **Demonstrates:** computer networks, socket programming.

6. Parking Lot / Elevator System (LLD) Month 7

Tech: C++/Java. **Features:** at least 3 design patterns applied (e.g., Strategy for pricing, Observer for notifications, Factory for object creation), full class diagrams. **Demonstrates:** SOLID principles, design patterns.

7. NLP Text Classifier Month 9

Tech: Python, Hugging Face `transformers`. **Features:** fine-tuned DistilBERT/BERT on a classification task, evaluation report (accuracy/F1/confusion matrix). **Demonstrates:** transformer fine-tuning basics.

8. RAG-Based Q&A Chatbot Month 10

Tech: LangChain/LlamaIndex, FAISS/Chroma, an LLM API, Streamlit/Gradio UI. **Features:** document chunking + embedding pipeline, similarity-search retrieval, grounded generation, simple chat interface. **Demonstrates:** retrieval-augmented generation end-to-end.

9. Fine-Tuned LLM + Deployment / Capstone Agentic App Months 11-12

Tech: Open-source LLM (Llama 3/Mistral) + LoRA, FastAPI, Docker, cloud deploy; capstone adds agent framework (LangGraph/tool-calling). **Features:** reproducible fine-tuning pipeline, containerized serving endpoint, and — for the capstone — a multi-step agentic assistant with a recorded demo. **Demonstrates:** the full LLM engineering stack, portfolio centerpiece.

Push every project to GitHub with a clear README (problem, approach, tech stack, how to run, screenshots/demo link). This becomes your interview talking-points library.

9. DSA Pattern Cheat Sheet

Pattern	Recognize When...	Example Problems
Two Pointers	Sorted array/list, pair/triplet sum, comparing from both ends	Two Sum II, 3Sum, Container With Most Water
Sliding Window	Contiguous subarray/substring with a condition (max/min length, sum, distinct chars)	Longest Substring Without Repeats, Max Sum Subarray of Size K
Fast & Slow Pointers	Linked list cycle, middle-of-list, "meeting point" problems	Linked List Cycle, Happy Number
Binary Search	Sorted (or monotonic) search space, "find minimum/maximum satisfying X"	Search in Rotated Array, Koko Eating Bananas, Binary Search on Answer
Merge Intervals	Overlapping ranges/intervals	Merge Intervals, Meeting Rooms II
Monotonic Stack	"Next greater/smaller element," histogram-style problems	Daily Temperatures, Largest Rectangle in Histogram
Top-K / Heap	"Kth largest/smallest," frequency-based top elements	Kth Largest Element, Top K Frequent Elements
BFS/DFS (Graph)	Connectivity, shortest path in unweighted graph, grid traversal	Number of Islands, Word Ladder, Rotten Oranges
Topological Sort	Dependency ordering, "course schedule" style	Course Schedule I/II, Alien Dictionary
Union-Find (DSU)	Dynamic connectivity, grouping, cycle detection in undirected graphs	Number of Provinces, Redundant Connection
0/1 Knapsack DP	Choose/skip items under a capacity constraint	Subset Sum, Partition Equal Subset Sum
Unbounded Knapsack DP	Unlimited reuse of items to hit a target	Coin Change, Rod Cutting
LCS-Family DP	Two-sequence comparison/alignment	Longest Common Subsequence, Edit Distance
Backtracking	"Generate all..." combinations/permutations/placements with constraints	N-Queens, Subsets, Word Search
Trie	Prefix search, autocomplete, word dictionary	Implement Trie, Word Search II
Bit Manipulation	"Single number," subsets via bitmask, power-of-two checks	Single Number, Subsets via Bitmask

10. System Design Building-Blocks Glossary

Term	One-Line Definition
Load Balancer	Distributes incoming requests across multiple servers to avoid overload (round-robin, least-connections, etc.)
Cache	Fast, temporary storage (e.g., Redis) placed in front of a slower data source to reduce latency and DB load
CDN	Geographically distributed servers that cache static content close to users to cut latency
Consistent Hashing	Hashing scheme that minimizes data movement when nodes are added/removed from a distributed system
Database Sharding	Splitting a large database horizontally across multiple machines by a shard key
Replication	Keeping copies of data on multiple nodes for availability/read-scaling (leader-follower or multi-leader)
CAP Theorem	A distributed system can only guarantee 2 of Consistency, Availability, Partition-tolerance at once
Message Queue	Asynchronous buffer (Kafka/RabbitMQ) decoupling producers and consumers of events/tasks
API Gateway	Single entry point that routes, authenticates, and rate-limits requests to backend microservices
Rate Limiting	Restricting request frequency per client (token bucket, sliding window log/counter algorithms)
Microservices	Architecture splitting an app into small, independently deployable services vs one monolith
Idempotency	Property where repeating the same request produces the same result — critical for safe retries
Eventual Consistency	System guarantees all replicas converge to the same state eventually, not instantly
Vector Database	Database optimized for similarity search over high-dimensional embeddings (FAISS, Pinecone)
Horizontal vs Vertical Scaling	Adding more machines (horizontal) vs adding more power to one machine (vertical)

11. OS / DBMS / CN Quick-Revision Cheat Sheet

Operating Systems

Term	One-Line Definition
Process vs Thread	Process = independent program with its own memory; thread = lightweight execution unit sharing process memory
Deadlock	Circular wait where processes hold resources the others need — requires mutual exclusion, hold-and-wait, no preemption, circular wait
Semaphore vs Mutex	Mutex = binary lock owned by one thread; semaphore = counter allowing N concurrent accesses
Paging	Memory management scheme splitting memory into fixed-size pages to avoid external fragmentation
Thrashing	System spends more time swapping pages than executing, due to insufficient physical memory
Context Switching	CPU saves state of one process/thread and loads another's — enables multitasking, has overhead

DBMS

Term	One-Line Definition
ACID	Atomicity, Consistency, Isolation, Durability — guarantees for reliable transactions
Normalization	Organizing tables to reduce redundancy (1NF → BCNF) by removing partial/transitive dependencies
Primary vs Foreign Key	Primary key uniquely identifies a row; foreign key references a primary key in another table
Index	Data structure (usually B-Tree) that speeds up lookups at the cost of extra writes/storage
Isolation Levels	Read Uncommitted → Read Committed → Repeatable Read → Serializable, trading concurrency for consistency
Joins	INNER (matching rows only), LEFT/RIGHT (all rows from one side + matches), FULL (all rows from both)

Computer Networks

Term	One-Line Definition
OSI Model	7-layer conceptual model: Physical, Data Link, Network, Transport, Session, Presentation, Application
TCP vs UDP	TCP = reliable, ordered, connection-based; UDP = fast, unordered, connectionless
Three-Way Handshake	SYN → SYN-ACK → ACK sequence that establishes a TCP connection
DNS	Translates human-readable domain names into IP addresses via a hierarchical lookup
HTTP vs HTTPS	HTTPS adds TLS/SSL encryption on top of HTTP for secure communication
NAT	Maps private internal IP addresses to a public IP for internet access

12. Note-Making System

- **Tool:** Obsidian or Notion — one "vault," organized by subject folders (DSA, OS, DBMS, CN, OOPs, System Design, LLM).
- **Format per topic:** Definition → Why it matters → Diagram/example → Common interview question → Gotchas/edge cases.
- **Anki flashcards:** convert every topic note into 3-5 spaced-repetition cards (definitions, complexity, trade-offs). Review 10-15 min daily — this is what makes 12 months of content actually stick.
- **DSA pattern sheet:** maintain a single living document mapping "if you see X constraint → try Y pattern" (see Section 9 for a starter version).
- **Weekly revision ritual (Sundays):** 30 minutes reviewing that week's notes and flagging weak areas for the following week.
- **Monthly "teach-back":** once a month, explain one hard topic out loud (or to a friend) as if teaching it — the Feynman technique exposes gaps fast.
- **Version your projects' READMEs** in the same vault so interview talking-points and code stay linked.

13. Typing Speed Improvement Plan

Month	WPM Target	Focus
1-2	30-40	Accuracy first (aim 95%+), proper finger placement — monkeytype.com / typing.com, 15-20 min/day
3-5	45-55	Speed drills + practice typing code snippets directly in your IDE
6-8	60-70	Sustained daily practice; try competitive typing tests weekly
9-12	75-90	Maintenance mode — 10 min/day is enough once the habit is built

Tip: once you pass ~50 WPM on prose, switch a portion of practice to typing actual code (symbols, brackets, indentation) since that's the real bottleneck in technical interviews and pair-programming rounds.

14. Behavioral & HR Interview Prep

STAR Method: structure every behavioral answer as **S**ituation → **T**ask → **A**ction → **R**esult. Keep answers to 60–90 seconds; lead with the result if asked to be brief.

Question Bank (prepare 1-2 stories that can flex to cover most of these)

- Tell me about a time you disagreed with a teammate or professor — how did you resolve it?
- Describe a project that failed or went wrong. What did you learn?
- Tell me about a time you had to learn something completely new under a deadline.
- Describe a situation where you had to prioritize among competing tasks.
- Tell me about a time you received tough feedback — how did you respond?
- Give an example of when you took initiative without being asked.
- Describe the most technically challenging project you've built (use one of your 9 projects).
- Tell me about a time you had to convince someone of your approach.
- Why do you want to work in AI/LLM engineering specifically?
- Where do you see yourself in the next 2–3 years?

Amazon-style prep: if applying there, map 1 story to each Leadership Principle you can recall (Customer Obsession, Ownership, Bias for Action, Dive Deep, Deliver Results, etc.) — these show up as explicit behavioral rounds.

15. Company-Wise Interview Focus

Company	Typical Focus
Google	Strong DSA (Medium/Hard), clean code, Googleyness/behavioral round, some system design for experienced tracks
Amazon	DSA rounds + heavy emphasis on Leadership Principles in every round, including technical ones
Meta	Fast, correct coding under time pressure, product-sense questions, "move fast" culture fit
Microsoft	DSA + practical problem-solving, design questions, collaborative/communication style valued
Apple	Deep technical rounds tied to the specific team, strong emphasis on fundamentals and attention to detail
Netflix / AI-focused startups	Fewer, deeper rounds; strong preference for demonstrated project depth (esp. ML/LLM) over pure LeetCode grinding

16. Resume, GitHub & Application Strategy

- **Resume:** one page, reverse-chronological, quantify impact (e.g., "reduced query time by 40%"), lead each project bullet with the action verb + outcome, not just tech used.
- **GitHub:** pin your 5 strongest projects, each with a README following the template in Section 8; keep commit history clean and consistent (looks better than a burst of commits the week before applying).
- **LinkedIn:** headline should state your target role + 1–2 standout skills (e.g., "Aspiring LLM Engineer | RAG · Fine-tuning · DSA"); post about your projects monthly to build visible progress.
- **Referrals first:** reach out to alumni/seniors at target companies before cold-applying — referral applications get dramatically higher response rates.
- **Application tracker:** keep a simple table (see below) and follow up politely after 7–10 days of silence.

Company	Role	Date Applied	Referral?	Status	Next Follow-up

--

17. Resource Library (Subject-wise)

Subject	Recommended Resources
C++ / DSA	Striver's A2Z DSA Sheet, Love Babbar Sheet, LeetCode, Codeforces, "Competitive Programmer's Handbook" (free PDF)
OOPs / LLD	GeeksforGeeks OOPs in C++, Refactoring.Guru (design patterns), "Head First Design Patterns"
Operating Systems	Gate Smashers / Neso Academy (YouTube), "Operating System Concepts" — Silberschatz/Galvin, OSTEP (free online book)
DBMS	Gate Smashers, "Database System Concepts" — Korth, LeetCode SQL, SQLZoo, Mode SQL Tutorial
Computer Networks	Neso Academy Computer Networks playlist, Kurose & Ross "Computer Networking: A Top-Down Approach"
System Design (HLD)	"Designing Data-Intensive Applications" — Kleppmann, Grokking the System Design Interview, ByteByteGo (YouTube + newsletter)
ML/DL Foundations	Andrew Ng's ML & DL Specializations (Coursera), fast.ai, "Hands-On ML with Scikit-Learn, Keras & TensorFlow"
NLP / Transformers	Hugging Face NLP Course, Jay Alammar's "The Illustrated Transformer" (blog), "Attention Is All You Need" paper
LLM Engineering	DeepLearning.AI short courses (LangChain, RAG, fine-tuning with PEFT), Full Stack Deep Learning / LLM Bootcamp materials, Hugging Face PEFT & TRL docs
Interview Prep	NeetCode (YouTube + site), Blind 75/150, Pramp, interviewing.io, "Cracking the Coding Interview"

18. Common Pitfalls to Avoid

- Grinding random LeetCode problems with no pattern tracking — always log which pattern each problem belongs to.
- Skipping the consolidation week "just this once" — this is exactly how topics silently decay.
- Starting a new project before the previous month's is at least functionally complete.
- Passive learning: watching tutorials without writing code or solving problems yourself.
- Ignoring typing practice once it feels "good enough" — 10 min/day maintenance is still worth it under interview time pressure.
- Neglecting behavioral prep until the week before interviews — start story-mapping in Month 11, not Month 12.
- Not writing README files for projects — an undocumented project is much weaker in interviews than a documented one.
- Over-indexing on LLM trends and under-indexing on DSA/fundamentals — FAANG bar-raisers still weight core CS heavily.

19. Monthly KPI Tracker

Metric	Track Every Month
DSA problems solved (cumulative)	___ / target
Typing WPM (current)	___ WPM
New topic theory completed	Yes / No
Project shipped & on GitHub	Yes / No
Anki review streak (days)	___ / 30
Weekly revision rituals completed	___ / 4
Mock interviews done (Phase 4 only)	___
Applications sent (Month 12 only)	___

Stay consistent over intense. Twelve months of steady daily reps beats any single heroic week. Good luck.